06. Low Level Operations

Kyu-haeng Lee

SW Convergence Coding II

Fall 2025



Bitwise Operators

- Bitwise operators operate on integer data at the bit level.

Operator		Description
&	bitwise AND	The bits in the result are set to 1 if the corresponding bits in the two operands are both 1.
|	bitwise inclusive OR	The bits in the result are set to 1 if at least one of the corresponding bits in the two operands is 1.
^	bitwise exclusive OR	The bits in the result are set to 1 if exactly one of the corresponding bits in the two operands is 1.
<<	left shift	Shifts the bits of the first operand left by the number of bits specified by the second operand; fill from the right with 0 bits.
>>	right shift	Shifts the bits of the first operand right by the number of bits specified by the second operand; the method of filling from the left is machine dependent.
~	one's complement	All 0 bits are set to 1 and all 1 bits are set to 0.

The result of bitwise operators is often machine dependent, that is, it depends on the size of integers on the local machine.

- Note: Bits are indexed from 0 starting from the right.

Bitwise Shift Operators

- <<
  - Shift the bits in an integer to the left.
  - Operands may be of any integer type (including char).
  - The result has the type of the left operand after promotion.

- >>
  - shift the bits to the right

Bitwise Shift Operators

- i << k
  - Shifts the bits in i to the left by k places
  - For each bit that is “shifted off” the left end of i, a 0 bit enters at the right.
- i >> k
  - Shifts the bits in i to the right by k places
  - If i is of an unsigned type or if the value of i is nonnegative, 0s are added at the left as needed.
  - If i is negative, the result is implementation-defined.

- Operands may be of any integer type, but use unsigned for portability

Bitwise Shift Operators

- unsigned short i, k;
  - i = 13; /* i is now 13 (binary 0000000000001101) */
  - k = i << 2; /* k is now 52 (binary 0000000000110100) */
  - k = i >> 2; /* k is now 3 (binary 0000000000000011) */

- To modify a variable by shifting its bits, use the compound assignment operators <<= and >>=:
  - i = 13; /* i is now 13 (binary 0000000000001101) */
  - i <<= 2; /* i is now 52 (binary 0000000000110100) */
  - i >>= 2; /* i is now 13 (binary 0000000000001101) */

Bitwise Complement, And, Exclusive Or, and Inclusive Or

- There are four additional bitwise operators:
  - ~ bitwise complement : unary
  - & bitwise and : binary
  - ^ bitwise exclusive or : binary
  - | bitwise inclusive or : binary
- The ~, &, ^, and | operators perform Boolean operations on all bits in their operands.
- The ^ operator produces 0 whenever both operands have a 1 bit, whereas | produces 1.

Bitwise Complement, And, Exclusive Or, and Inclusive Or

- Examples of the ~, &, ^, and | operators:
  - unsigned short i, j, k;

  - i = 21; /* i is now 21 (binary 0000000000010101) */
  - j = 56; /* j is now 56 (binary 0000000000111000) */

  - k = ~i; /* k is now 65514 (binary 1111111111101010) */
  - k = i & j; /* k is now 16 (binary 0000000000010000) */
  - k = i ^ j; /* k is now 45 (binary 0000000000101101) */
  - k = i | j; /* k is now 61 (binary 0000000000111101) */

Bitwise Complement, And, Exclusive Or, and Inclusive Or

- The compound assignment operators:

```
i = 21;     /* i is now 21 (binary 0000000000010101) */
j = 56;     /* j is now 56 (binary 0000000000111000) */

i &= j;     /* i is now 16 (binary 0000000000010000) */
i ^= j;     /* i is now 40 (binary 0000000000101000) */
i |= j;     /* i is now 56 (binary 0000000000111000) */
```

Precedence

- The bitwise shift operators have lower precedence than the arithmetic operators, which can cause surprises:
  - i << 2 + 1 means i << (2 + 1), not (i << 2) + 1

Machine Dependency

- The result of bitwise operators is often machine dependent, that is, it depends on the size of integers on the local machine.

- The ~ operator can be used to help make low-level programs more portable.
  - An integer whose bits are all 1: ~0
  - An integer whose bits are all 1 except for the last five: ~0x1f

Using the Bitwise Operators to Access Bits

- The bitwise operators can be used to extract or modify data stored in a small number of bits.
- Common single-bit operations:
  - Setting a bit
  - Clearing a bit
  - Testing a bit
- Assumptions:
  - i is a 16-bit unsigned short variable.
  - The leftmost—or most significant—bit is numbered 15 and the least significant is numbered 0.

Using the Bitwise Operators to Access Bits

- Setting a bit.
  - i = 0x0000; /* i is now 0000000000000000 */
  - i |= 0x0010; /* i is now 0000000000010000 */
- If the position of the bit is stored in the variable k, a shift operator can be used to create the mask:
  - i |= 1 << k; /* sets bit k */
  - The constant used to set a bit is known as a mask.
- Example:
  - If k has the value 3, then 1 << k is 0x0008.

Using the Bitwise Operators to Access Bits

- Clearing a bit.
  - i = 0x00ff; /* i is now 0000000011111111 */
  - i &= ~0x0010; /* i is now 0000000011101111 */

- A statement that clears a bit whose position is stored in a variable:
  - i &= ~(1 << k); /* clears bit k */

Using the Bitwise Operators to Access Bits

- Testing a bit.

- An if statement that tests whether bit 4 of i is set:
  - if (i & 0x0010) … /* tests bit 4 */

- A statement that tests whether bit k is set:
  - if (i & 1 << k) … /* tests bit k */

enum and Bit Masks

- Consider the names that represent the three bit positions:
  - enum {BLUE = 1, GREEN = 2, RED = 4};

- Examples of setting, clearing, and testing the BLUE bit:
  - i |= BLUE; – sets the BLUE bit
  - i &= ~BLUE; – clears the BLUE bit
  - if (i & BLUE) – tests the BLUE bit

enum and Bit Masks

- Consider the names that represent the three bit positions:
  - enum {BLUE = 1, GREEN = 2, RED = 4};

- It's also easy to set, clear, or test several bits at time:
  - i |= BLUE|GREEN – sets the BLUE and GREEN bits
  - i &= ~(BLUE|GREEN) – clears BLUE and GREEN
  - if (i&(BLUE|GREEN)) – tests BLUE and GREEN
    - The if statement tests whether either the BLUE bit or the GREEN bit is set.

Bit Fields

- A group of several consecutive bits is a bit-field.

- Common bit-field operations:
  - Modifying a bit-field
  - Retrieving a bit-field

Bit Fields

- Modifying a bit-field
  - A bitwise and (to clear the bit-field)
  - A bitwise or (to store new bits in the bit-field)

  - Example: stores 101 in bits 4-6
    - i = i & ~0x0070 | 0x0050;
    - The & clears bits 4-6 and the | sets bits 4 and 6
    - Just using | will not always work, as it doesn't clear bit 5

  - Assume that k contains the value to be stored in bits 4–6 of i. To store k into position 4-6 of i:
    - i = (i & ~0x0070) | (k << 4);

Bit Fields

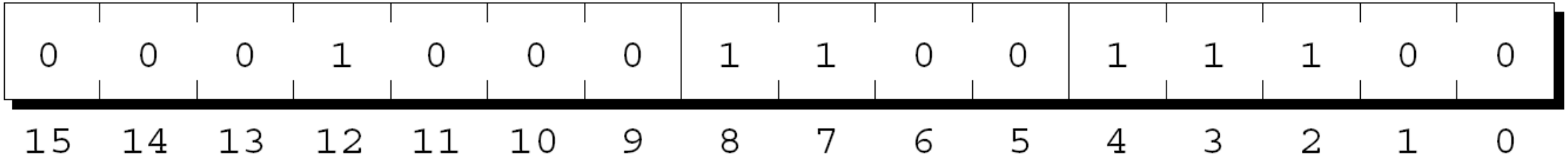
- Retrieving a bit-field
  - Fetching a bit-field at the right end of a number (in the least significant bits)
    - Example: retrieve bits 0-2 of i
      - j = i & 0x0007;
  - What if the bit-field isn't at the right end of i?
    - Example: retrieve bits 4-6 of i
      - First shift the bit-field to the end
      - Then extracting the field using the & operator:
    - j = (i >> 4) & 0x0007;

Bit-Fields in Structures

- Structure declarations whose members are bit-fields.
- 16 bits in total for a date, with 5 bits for the day, 4 bits for the month, and 7 bits for the year:

```
struct file_date {
    unsigned int day : 5;
    unsigned int month : 4;
    unsigned int year : 7;
}
```

```
struct file_date fd;
fd.day = 28;
fd.month = 12;
fd.year = 8;
```



Bit-Fields and Memory

- Bit Fields do not have addresses
  - scanf("%d", &fd.day); /*wrong*/

```
struct file_date {
    unsigned int day : 5;
    unsigned int month : 4;
    unsigned int year : 7;
}
```

- How bit fields are stored is highly machine and implementation dependent. The example in the previous slide assumes 16-bit units.
- When bit fields do not fit a storage unit precisely, what happens is compiler dependent.

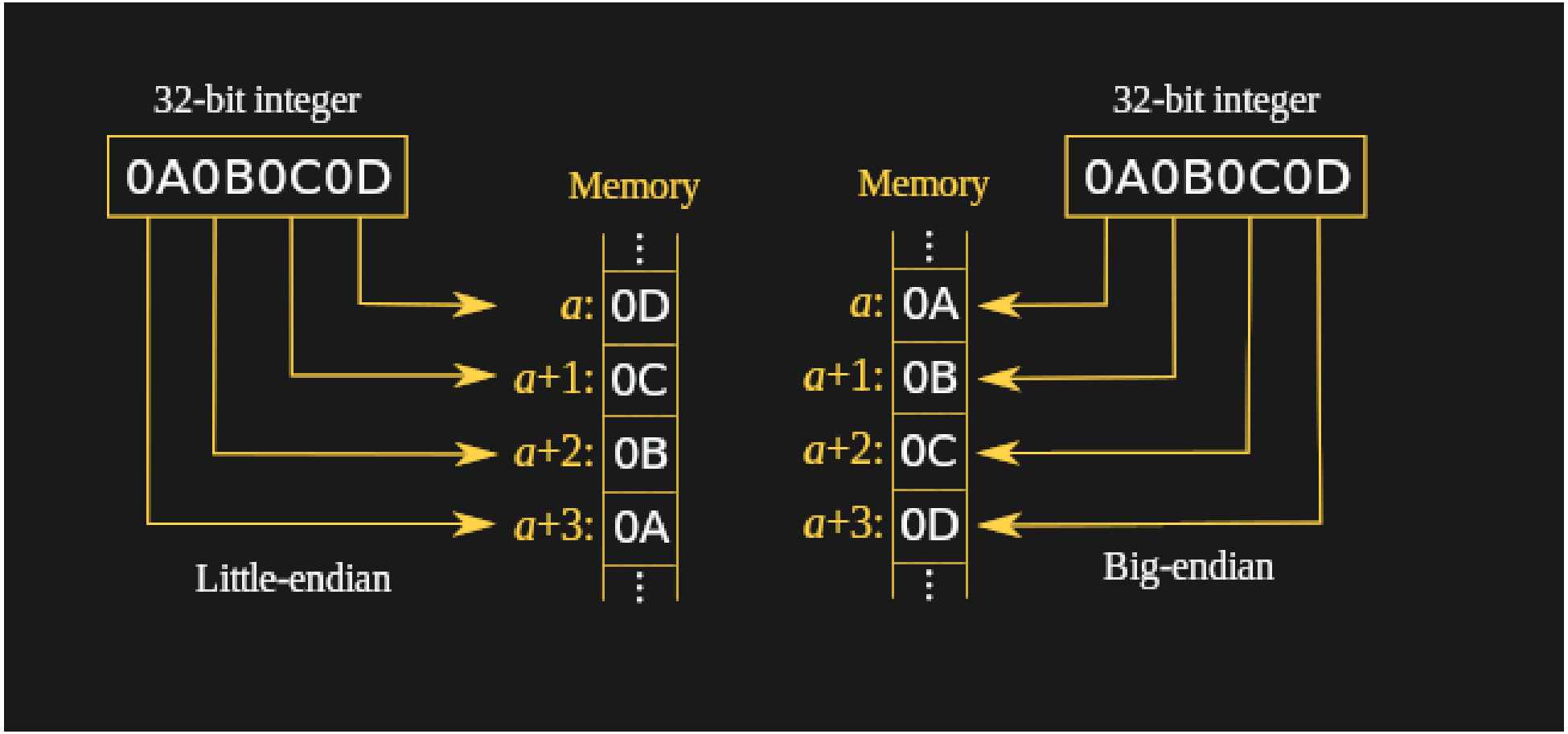
Big-endian and Little-endian

- The order of storing bytes
  - Two ways to store more than 2B data in memory

  - Big-endian: Bytes are stored in “natural” order (the leftmost byte comes first).
  - Little-endian: Bytes are stored in reverse order (the leftmost byte comes last).
- Example
  - x86 processors use little-endian order.
  - Network data is sent to in big-endian.

- Don’t normally need to worry about byte ordering.
  - However, programs that deal with memory at a low level must be aware of the order in which bytes are stored.

Big-endian and Little-endian



https://hackaday.com/2020/08/04/dont-let-endianness-flip-you-around/

Big-endian and Little-endian

- A way to determine endianness of your machine

```c
#include <stdio.h>

int main()
{
    unsigned int i = 1;
    char* c = (char*)&i;
    if (*c)
        printf("Little endian");
    else
        printf("Big endian");
    return 0;
}
```

References

- Chapter 10. C Structures, Unions, Bit Manipulation and Enumerations, C: How to Program, Paul Deitel et al.